IT342 – System Integration and Architecture

MedBuddy

Project Development Phase 1

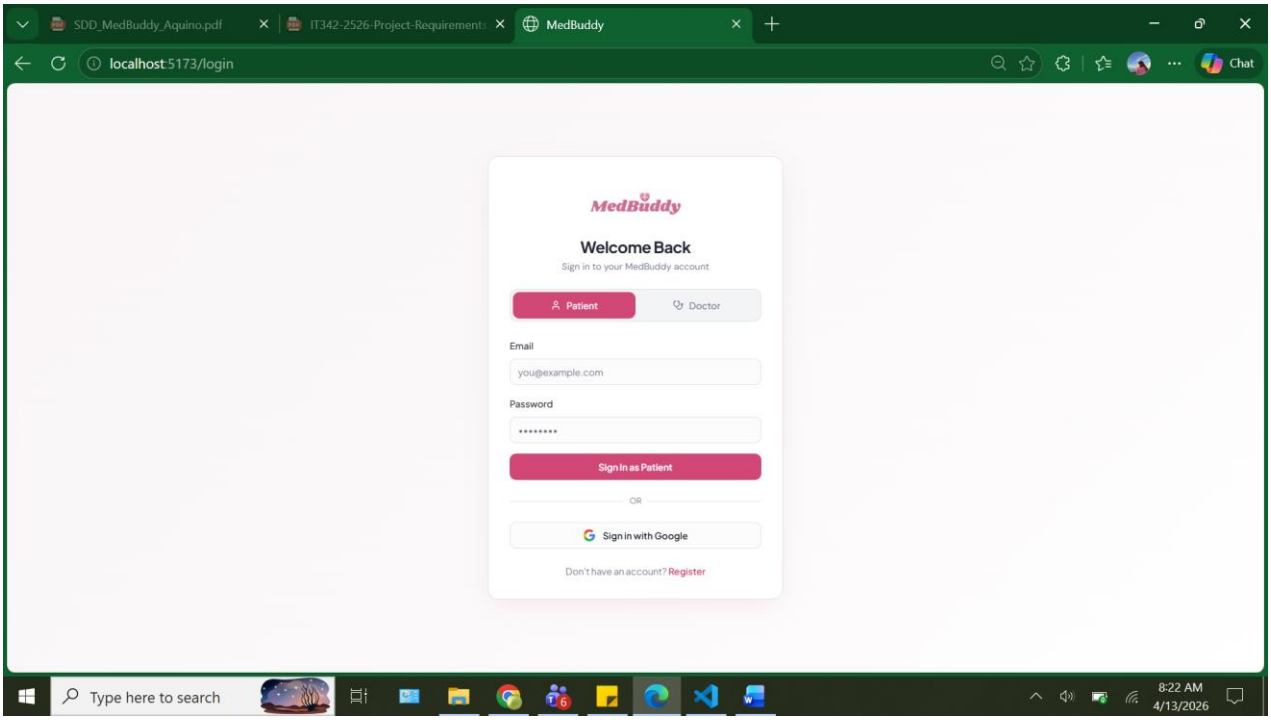
Third-Party Authentication Integration

Prepared by	Aquino, Julianne Rose P.
Section	IT342-G2
Project Name	MedBuddy
Github link	https://github.com/oblivjules/IT342_MedBuddy_Aquino.git
Branch	feature/google-oauth2
Date	April 13, 2026
Phase	Phase 1 – Third-Party Authentication Integration
Status	Completed

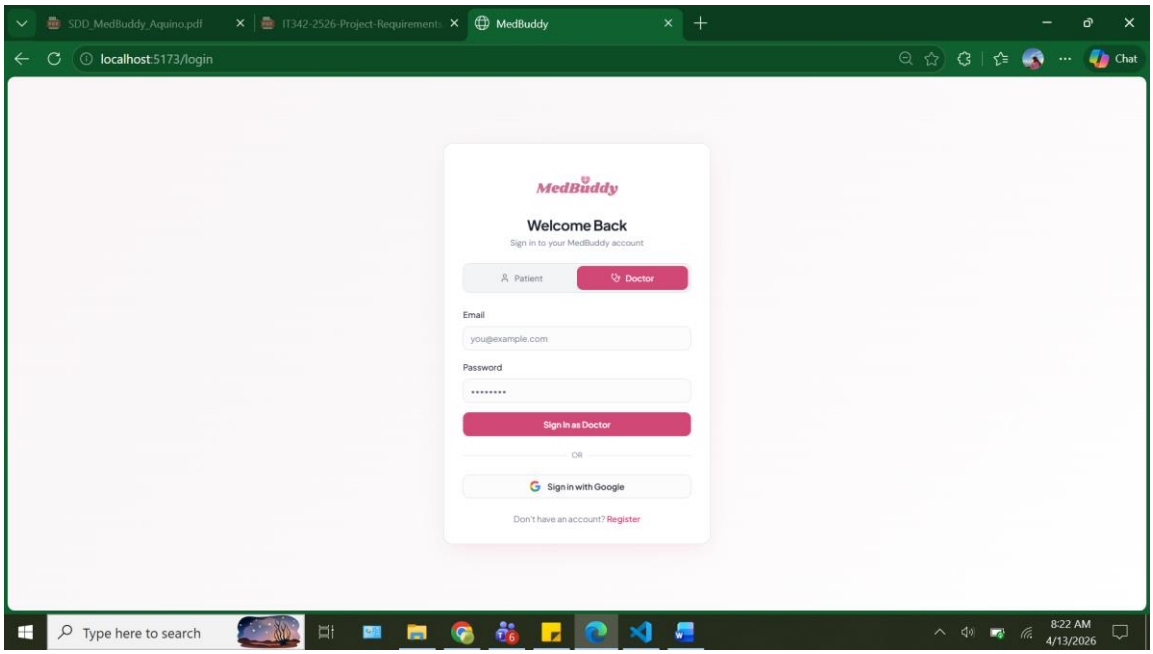
1. Screenshots of Implementation

- **Screenshot 1 – Third-party sign-in button in the frontend**

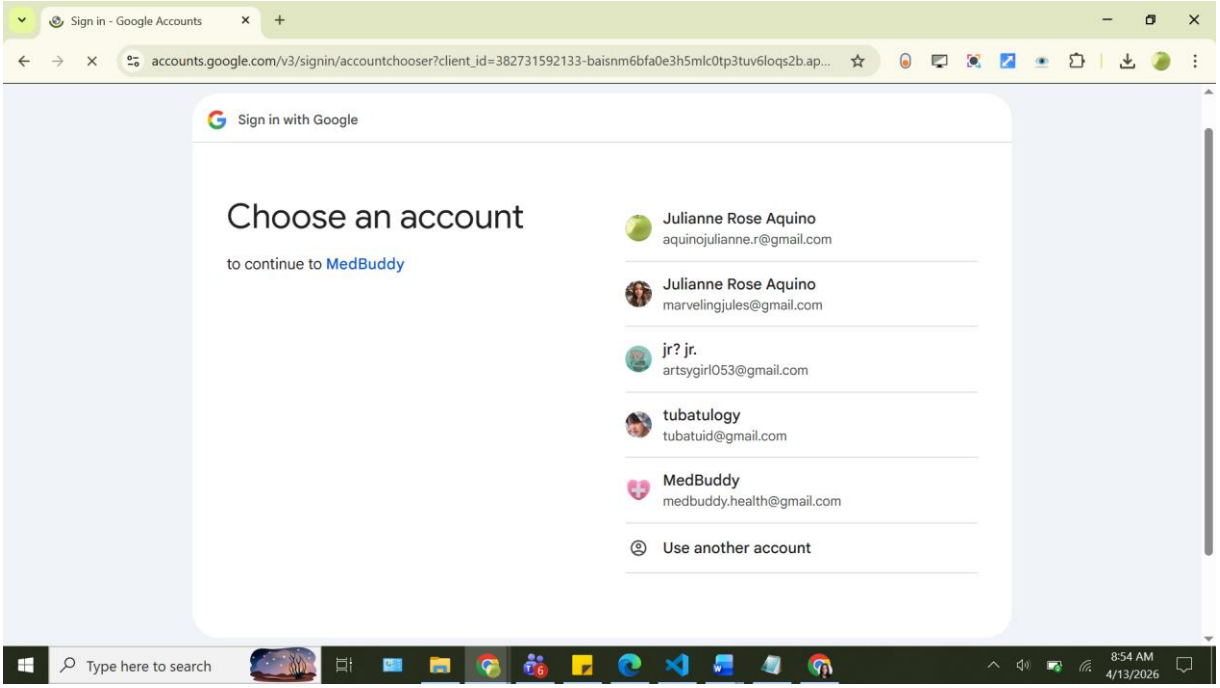
  a. Patient Sign-in Page



  b. Doctor Sign-in Page

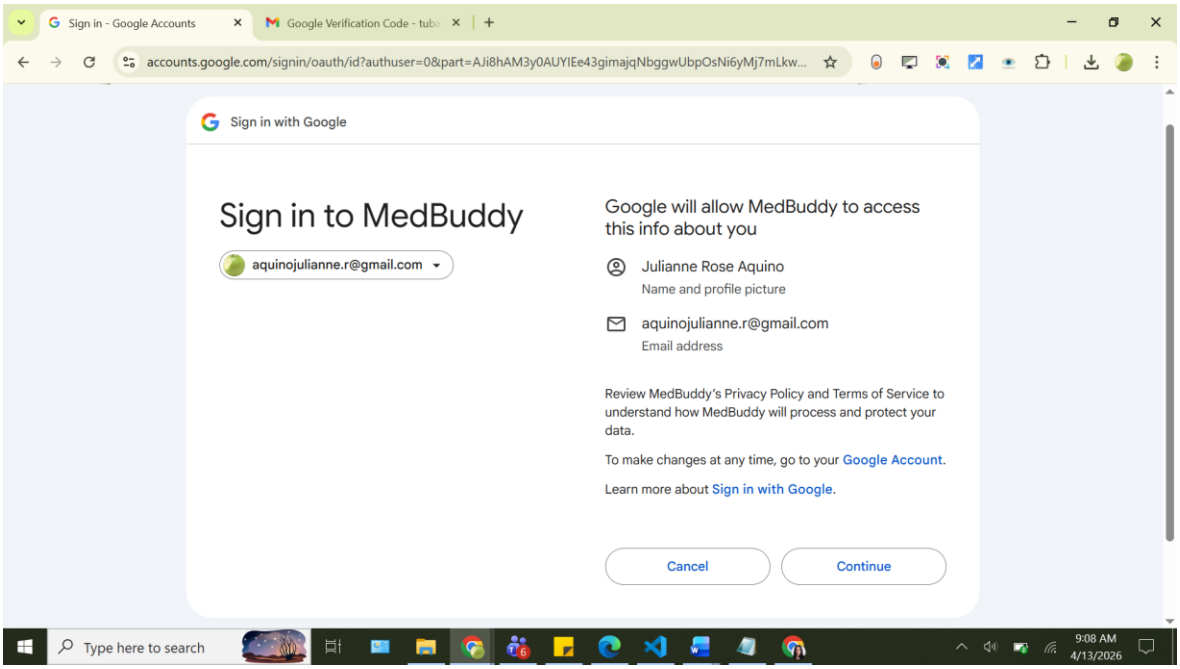


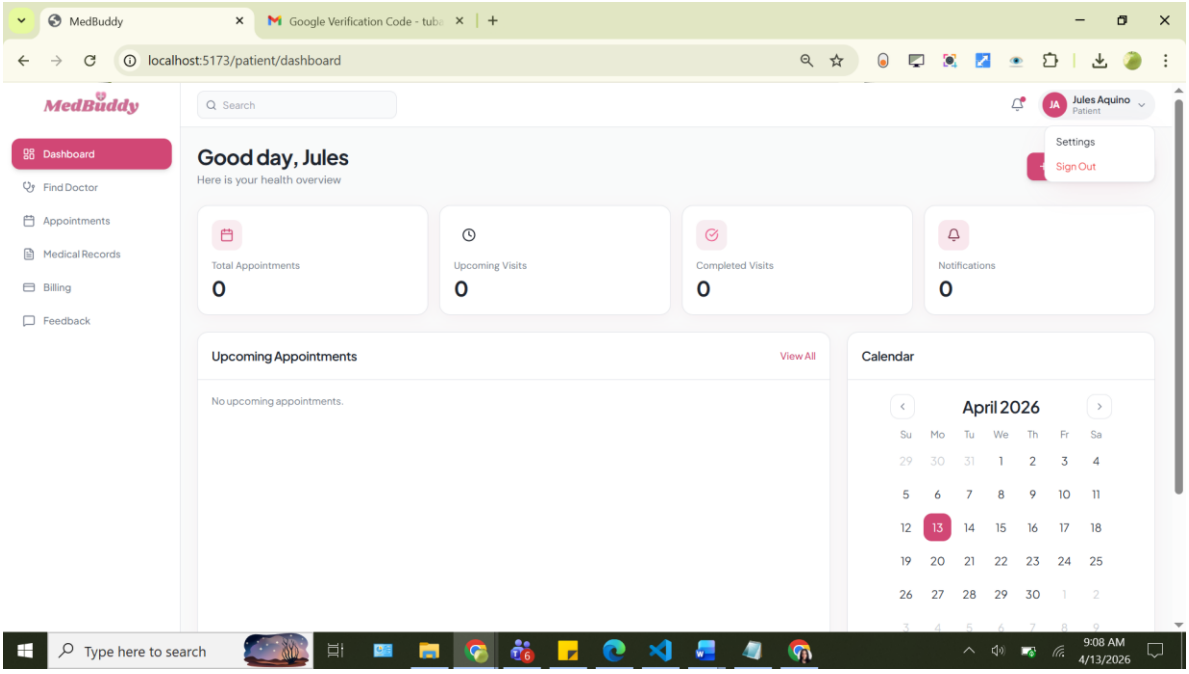
Screenshot 2 – Provider account selection or authentication page



Successful login result in the system

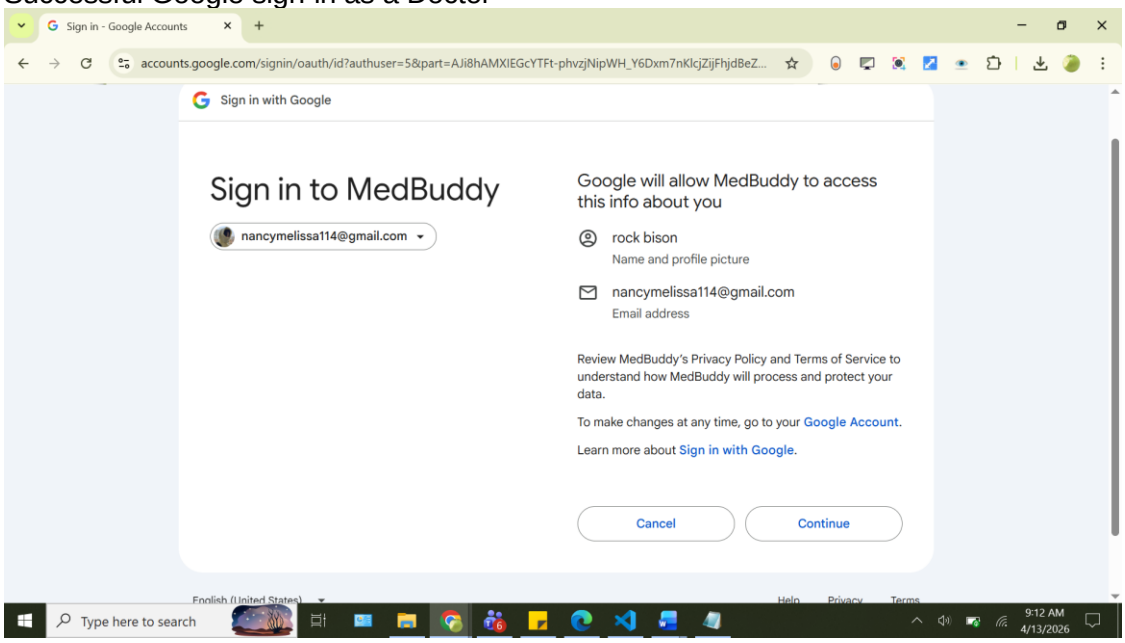
a. Successful Google sign-in as a Patient

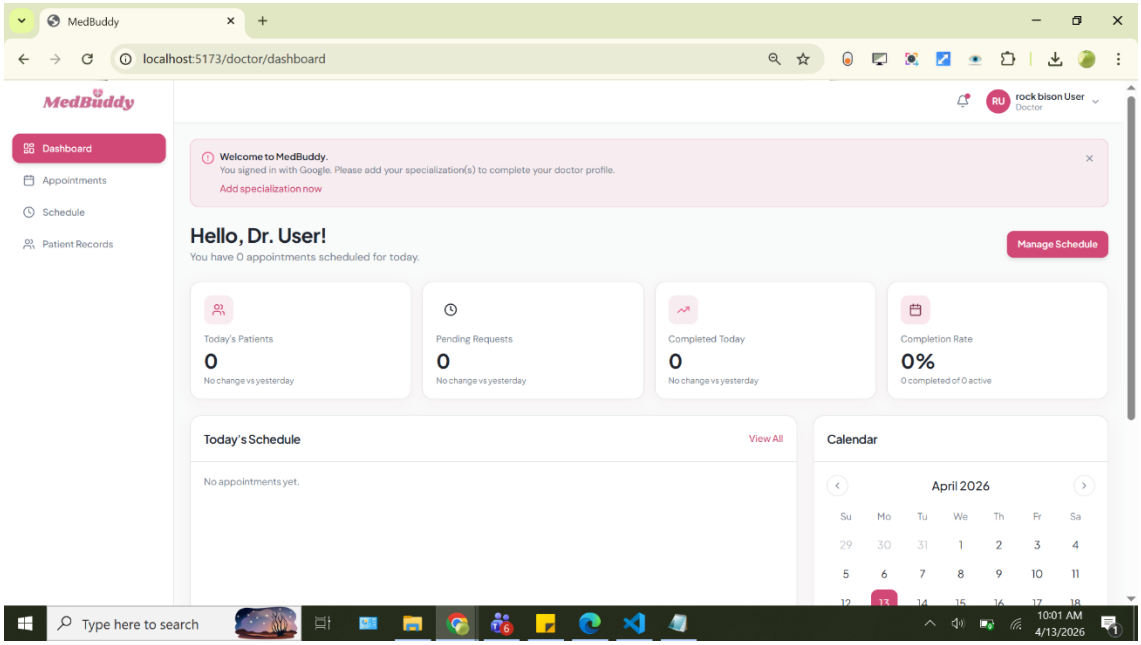




After completing Google authentication, the user is redirected to the Patient Dashboard.

b. Successful Google sign-in as a Doctor





After completing Google authentication, the user is redirected to the Doctor Dashboard with a welcome message requiring them to add a specialization/s, an important step that was skipped due to Google sign-in.

Screenshot 5 – Database Record of New User

a. New Patient Record

#	id	created_at	email	password_hash	provider	role	profile
1	24	2026-03-28 15:21:31.205867	julianneaquino.aica@gmail.com		GOOGLE	PATIENT	
2	1	2026-03-09 09:25:31.116534	aquinojulianne.r@gmail.com		GOOGLE	PATIENT	

b. New Doctor Record

#	id	created_at	email	password_hash	provider	role	profile_image_url
1	42	2026-04-13 09:40:46.777768	nancymelissa114@gmail.com		GOOGLE	DOCTOR	

3. Implementation Summary

MedBuddy integrates Google Sign-In as a third-party OAuth2 provider, allowing both Patient and Doctor users to authenticate using their existing Google accounts. The integration works alongside the existing JWT-based username/password login flow, ensuring backward compatibility with all previously registered users.

3.1 Third-Party Authentication Provider

Provider	Google (OAuth 2.0 / OpenID Connect)
Protocol	OAuth 2.0 Authorization Code Flow with PKCE
Scope	openid, profile, email
Token standard	Google ID Token (JWT)

3.2 Backend Implementation (Spring Boot)

The backend was extended to support Google OAuth2 through the following steps:

Spring Security & OAuth2 Configuration

- spring-boot-starter-oauth2-client was added to pom.xml to enable OAuth2 client support within Spring Security.
- SecurityConfig.java was updated to permit the /api/auth/google endpoint publicly and apply JWT filter protection to all other protected routes.
- Google OAuth2 client credentials (Client ID and Client Secret) were configured in application.properties, with the redirect URI pointing to the appropriate callback endpoint.

Google Token Verification

- A dedicated GoogleAuthController exposes a POST /api/auth/google endpoint that receives the Google ID Token from the frontend.
- The token is verified server-side using the Google API Client Library (google-api-client), extracting the user's email, name, and Google subject ID (sub).
- Token verification ensures the audience (aud) matches the application's registered Client ID, preventing token misuse.

User Provisioning & Role Assignment

- Upon successful Google token verification, the backend checks whether a User record already exists for the verified email address.
- If no record exists, a new User is created with the googleId field populated, the displayName sourced from the Google profile, and the role set to PATIENT by default (Doctor accounts still require full registration due to specialization requirements).
- Existing users who previously registered with email/password can also sign in with Google, provided their email addresses match.

JWT Issuance

- After user lookup or creation, the backend issues a standard MedBuddy JWT token using the same JwtService already in use for regular login.
- The response body is an AuthResponseDto containing the JWT and the user object, identical in structure to the regular login response, ensuring the frontend handles both flows uniformly.

3. Implementation Summary

3.3 Frontend Integration (ReactJS)

The web frontend was updated to surface the Google sign-in option and handle the OAuth2 credential flow:

- @react-oauth/google (v0.12+) was installed and the application root was wrapped in GoogleOAuthProvider, configured with the Google Client ID.
- A “Sign in with Google” button was added to both the Patient Sign-in Page and the Doctor Sign-in Page using the GoogleLogin component. The button renders with Google’s standard branding (logo, label, and color) in compliance with Google’s identity guidelines.
- On successful credential selection, the onSuccess callback receives a credential object containing the ID token. This token is immediately forwarded to the backend via POST /api/auth/google.
- On receipt of the AuthResponseDto from the backend, the JWT is stored in the application’s token manager (localStorage with the same key used for regular login) and the user is routed to the appropriate dashboard based on their role (Patient Dashboard or Doctor Dashboard).
- On authentication failure (either Google-side or backend-side), a user-visible error toast is displayed. The onError callback from GoogleLogin handles provider-side failures, while HTTP error responses from the backend are caught and surfaced as descriptive messages.

3.4 Backend Technologies & Libraries

Technology / Library	Purpose
Spring Boot 3 + Spring Security	Core framework; security filter chain configuration
spring-boot-starter-oauth2-client	OAuth2 client support within Spring Security
google-api-client (com.google.api-client)	Server-side Google ID Token verification
JJWT (io.jsonwebtoken)	JWT issuance and validation for MedBuddy sessions
Spring Data JPA + MySQL	User persistence; googleId column added to users table
Lombok	Reduces boilerplate on entity and DTO classes

4. Challenges Encountered

Challenge	Resolution
Role assignment for Google-authenticated users	Since Google does not provide a user role, PATIENT is assigned by default. Doctor accounts continue to require full registration to capture specialization data.
Handling existing users who sign in with Google for the first time	The backend checks by email before creating a new record, allowing existing email/password users to also use Google Sign-In without creating duplicate accounts.
CORS configuration for the React frontend OAuth callback	Spring Security's CORS configuration was updated to allow the frontend origin (localhost:5173 in dev) for the /api/auth/google endpoint.
Keeping both auth flows in sync in the token manager	The JWT returned from Google login is stored using the same TokenManager key as regular login, so all subsequent API calls remain authenticated without any code path duplication.